# Data Collection: APIs & Web Scraping

**SOCI 3238 Digital Sociology · Tutorial 3**

Nov 9, 2026 (Mon) 12:30–14:00 · Sino Building 408B

Xuanlong QIN (Teaching Assistant)

# Agenda

1. Where are we? From built-in data to the open web
2. Part I · APIs — the structured door to web data
    - How the web works: requests & JSON
    - GitHub API & OpenAlex API in R (`httr2`)
    - API keys, rate limits & manners

# Agenda

3. Part II · Web scraping — when there is no API
    - HTML & CSS selectors
    - Scraping with `rvest`
    - Ethics: robots.txt, politeness, legality
4. Hands-on

# Where are we?

**Tutorial 1** — the toolbox

- Variables, data structures, functions, plots

**Tutorial 2** — putting the toolbox to work

- Hypothesis tests, regression models, networks

# Where are we?

**Tutorial 3** — where does the data come from?

- So far: built-in datasets (`mtcars`, karate club)
- Today: collect data from the open web yourself

Two doors to web data:

- **API** — the site's official door for programs (structured data)
- **Web scraping** — reading the page built for humans (HTML)

# Part I · APIs

# Why data collection?

- Real research questions rarely match built-in datasets
- Digital sociology data lives on the web:
  - Social media posts, followers, retweets
  - Scholarly databases (papers, citations)
  - Open-source communities (GitHub)
  - Government & organizational portals
- Two ways in: **API** (if the site offers one) or **scraping**

> Rule of thumb: use the API when there is one

# How the web works

- Your computer (**client**) sends an **HTTP request** to a **server**
- The server sends back a **response**
- Two languages of the web:
  - **HTML** — built for humans, rendered by browsers
  - **JSON** — built for programs, structured data

```
You/local (R)  ── request ──▶  Server  ── response ──▶  You/local (R)
        "give me X"              data or page
```

# What is an API?

**API** = Application Programming Interface

- An official door the website provides **for programs**
- You ask for data → the server returns structured **JSON**
- No need to "read" a webpage like a human

# What is JSON?

**JSON** = JavaScript Object Notation

```
{
  "full_name": "igraph/rigraph",
  "stargazers_count": 195,
  "language": "R"
}
```

- Key–value pairs, nested like R lists
- `{ }` = object (like a list) · `[ ]` = array (like a vector)
- Note: JSON is one common data **format**, not the only one — industry likes it because it is clear, standard and easy to read

# Calling an API from R: httr2

```
library(httr2)

resp <- request("https://api.github.com/repos/igraph/rigraph") |>
  req_perform()

repo <- resp |> resp_body_json()
repo$stargazers_count
```

- `request()` — build the request
- `req_perform()` — send it, get the response
- `resp_body_json()` — turn the JSON into an R list

# Read the dataflow (1): nested functions

Nested calls must be read **inside-out** — hard to follow:

```
result <- mean(abs(sqrt(c(-4, 9, 16))))
```

`abs()` of `sqrt()` of a vector, then `mean()` of the result

# Read the dataflow (2): the pipe

The same computation, read **left-to-right**:

```
result <- c(-4, 9, 16) |>
  sqrt() |>
  abs() |>
  mean()
```

- Each line is one step: data → sqrt → abs → mean → result
- If nested functions feel confusing, write **one step per line** with the pipe

# Example 1: GitHub API (no key needed)

Social data: open-source communities

```
resp <- request("https://api.github.com/search/repositories") |>
  req_url_query(q = "sociology", sort = "stars") |>
  req_perform()

repos <- resp |> resp_body_json()
repos$total_count   # how many sociology repos exist?
```

- Query parameters: `q` (search term), `sort`, `per_page`
- `items` holds the results — one list per repository
- Extract fields with `sapply()` → data frame (Tutorial 1!)

# Example 2: OpenAlex API (no key needed)

Free scholarly database — papers, authors, citations

```
resp <- request("https://api.openalex.org/works") |>
  req_url_query(search = "digital sociology", per_page = 10) |>
  req_perform()

works <- resp |> resp_body_json()
works$meta$count        # total matching works
works$results           # the first 10 works
```

- Very relevant for social science: find literature, track citations
- Nested JSON → navigate with `$` and `[[ ]]`
- Same logic as GitHub — learn one, use many

# JSON → data frame

```
title  <- sapply(works$results, function(w) w$title)
year   <- sapply(works$results, function(w) w$publication_year)
cites  <- sapply(works$results, function(w) w$cited_by_count)

papers <- data.frame(title = title, year = year, citations = cites)
papers
```

> The API gives you a list of lists; `sapply()` turns it into a table

# API keys & rate limits

- Some APIs require a **key** (Twitter/X, YouTube, NewsAPI)
  - Register for a key → keep it **secret** (never share scripts containing keys!)
- **Rate limits**: free APIs cap how often you can ask
  - GitHub: 60 requests/hour without a key; if you hit the limit → error 403
- **Manners**:
  - Identify yourself (User-Agent / contact email)
  - Cache results — don't re-download what you already have
  - `Sys.sleep()` between requests in a loop

# Part II · Web Scraping

# When there is no API

- Most websites offer no API
- But the data is there — inside the **HTML** the browser renders
- **Web scraping** = fetch the page + pull out the pieces you need
- The webpage itself becomes your dataset

> Analogy: API = a served meal; scraping = cooking from the raw ingredients yourself

# How a webpage is built

```
<article class="product_pod">
  <h3><a href="..." title="A Light in the Attic">A Light in the...</a></h3>
  <p class="price_color">£51.77</p>
  <p class="star-rating Three">...stars...</p>
</article>
```

- **HTML** = structure: tags like `<h3>`, `<a>`, `<p>`, `<table>`
- **CSS** = appearance; classes/ids are the hooks we select with
- Scraping = fetch HTML → select elements → extract text / attributes
- **HTML syntax**: https://www.w3schools.com/html/html_intro.asp

# CSS selectors (cheat sheet)

| Selector | Meaning | Example |
|---|---|---|
| `h3` | all `<h3>` tags | titles |
| `.price_color` | any element with this class | prices |
| `#main` | the element with this id | one element |
| `h3 a` | `<a>` inside `<h3>` | links in titles |
| `p.star-rating` | `<p>` with that class | ratings |

How to find them:

- Browser: right-click → Inspect (DevTools, F12)
- SelectorGadget browser extension

# rvest: the scraping toolbox

```
library(rvest)

html <- read_html("http://books.toscrape.com/")

html |> html_elements("h3 a") |> html_attr("title")
html |> html_elements("p.price_color") |> html_text()
```

- `read_html()` — fetch & parse the page
- `html_elements()` — select by CSS selector
- `html_text()` — the visible text
- `html_attr()` — an attribute (e.g. `title`, `href`)
- `html_table()` — whole `<table>`s at once

# Example: books.toscrape.com

A website built **for practicing scraping** — safe to play with

Goal: all book titles, prices & ratings on page 1

```
titles  <- html |> html_elements("h3 a") |> html_attr("title")
prices  <- html |> html_elements("p.price_color") |> html_text()
ratings <- html |> html_elements("p.star-rating") |> html_attr("class")
ratings <- gsub("star-rating ", "", ratings)   # clean up

books <- data.frame(title = titles, price = prices, rating = ratings)
head(books)
```

# From scraped text to numbers

```
books$price <- as.numeric(gsub("£", "", books$price))

mean(books$price)
table(books$rating)
```

- Scraped data is always **text** first
- Clean it: `gsub()`, `as.numeric()`, `trimws()` …
- Then the Tutorial 1 & 2 tools apply: plots, models, networks

# Pagination: scraping several pages

Books are spread over 50 pages — loop over page URLs (Attention)

```
books_all <- data.frame()

for (i in 1:3) {
  url <- if (i == 1) "http://books.toscrape.com/" else
         paste0("http://books.toscrape.com/catalogue/page-", i, ".html")
  page <- read_html(url)
  Sys.sleep(1)          # be polite: pause between requests
  # ... extract titles, prices, ratings into page_df ...
  books_all <- rbind(books_all, page_df)
}
```

- `paste0()` builds the URL — it's just a string, so we can loop over it
- **Always** `Sys.sleep()` between requests to the same server

# Dynamic pages (advanced)

Some pages are built by JavaScript *after* loading — `read_html()` sees nothing

```
<div id="app"></div>   <!-- empty! content comes later -->
```

Solutions:

1. Find the API the page itself calls (DevTools → Network tab) — often the easiest
2. Drive a real browser: `chromote`, `playwright` packages
3. `RSelenium` for automating a browser session

> For today: prefer sites that render in plain HTML

# Ethics & legality of scraping

- Check **robots.txt** first: `books.toscrape.com/robots.txt`
- Respect the site's **Terms of Service** — some forbid scraping
- **Don't hammer** the server: `Sys.sleep()`, cache, reuse what you downloaded
- Scraped data ≠ free data: copyright & personal data protections still apply
- Same research ethics as any data collection (privacy, consent, purpose)
- When in doubt: ask the site owner, or use an official API/dataset

> A scraper is just a visitor; be the polite kind
> My experience: a student at UCSD got in trouble over scraping data

# Hands-on

Part A · APIs (10 min)
Part B · Web scraping (10 min)

# Hands-on A: APIs

1. **GitHub** — search repositories for "sociology", sorted by stars
   - Which repository has the most stars? Which language is most common?
2. **OpenAlex** — fetch 10 works matching "digital sociology"
   - Build a data frame of title, year, citations
   - Which paper is the most cited? What is the average year?

# Hands-on B: Web scraping

1. Scrape **books.toscrape.com page 1**: title, price, rating → data frame
2. Extend to **pages 1–3** with a loop (remember `Sys.sleep()`!)
3. With the combined data frame:
   - Average price?
   - Most common rating?
   - Price vs rating — any pattern?

> Network slow or blocked? Use the cached copies in the materials folder (`books_page1.html`, `openalex_digital_sociology.json`, `github_sociology_repos.json`)

# For next time

- Tutorial 4: Text Analysis (Advanced) — Nov 23
- Everything we collect today becomes input for:
  - Models & networks (Tutorial 2)
  - Text analysis (Tutorial 4)

# Materials for your references

- HTML: https://www.w3schools.com/html/html_intro.asp
- WebScraping in R: https://statsandr.com/blog/web-scraping-in-r/
- Use an LLM (ChatGPT/Claude) to help you write scraping code

# Thanks!

Slides & materials: course content on Blackboard or download from Github.

Questions → email (xuanlong@link.cuhk.edu.hk)